

# **MLOps Assignment 3**

End-to-End Hugging Face Model Training & Docker Deployment

Fine-Tuning a Transformer for Book-Genre Classification

Laksh Mendpara

Roll No: B23CS1037

February 2026

## **Contents**

# 1 Introduction

This report documents the end-to-end machine learning workflow for Assignment 3. The goal is to fine-tune a pre-trained language model to classify book reviews into one of eight genres using the GoodReads dataset, containerise the entire pipeline with Docker, and publish the trained model to the Hugging Face Hub.

**GitHub Repository:** <https://github.com/Laksh-Mendpara/MLOps-Laksh-B23CS1037/tree/Assignment-3>

**Hugging Face Model:** <https://huggingface.co/Laksh-Mendpara/MLOps-Assignment-3>

## 2 Model Selection

### 2.1 Chosen Model: roberta-base

After initial experimentation with `distilbert-base-cased`, a switch was made to `roberta-base` to improve classification accuracy. Table ?? compares the candidate models.

Table 1: Candidate model comparison

| Model                      | Params | Size   | Approx. Acc. | Extra deps |
|----------------------------|--------|--------|--------------|------------|
| distilbert-base-cased      | 66 M   | 260 MB | ~60%         | none       |
| <b>roberta-base</b>        | 125 M  | 480 MB | ~62%+        | none       |
| microsoft/deberta-v3-small | 140 M  | 278 MB | ~74%         | protobuf   |

**Rationale for roberta-base:**

- RoBERTa was trained with a more robust regime (dynamic masking, no NSP, larger batches) compared to original BERT.
- It does not require additional system libraries (unlike DeBERTa-v3 which needs `protobuf` for its `SentencePiece` tokenizer).
- It is available out-of-the-box with `AutoTokenizer` and `AutoModelForSequenceClassification`.
- `microsoft/deberta-v3-small` is documented as the recommended upgrade path once `protobuf` is added to the Docker image.

## 3 Dataset

The dataset is drawn from the [UCSD GoodReads](#) corpus. Reviews are streamed directly from eight per-genre gzip-compressed JSON files.

Reviews are cached locally as a pickle file after the first download so that re-runs do not require a network connection.

## 4 Training Summary

Training was conducted with the Hugging Face `Trainer` API inside a Docker container with GPU acceleration (`--gpus all`).

Table 2: Dataset statistics

| Genre                  | Train        | Test         |
|------------------------|--------------|--------------|
| children               | 800          | 200          |
| comics_graphic         | 800          | 200          |
| fantasy_paranormal     | 800          | 200          |
| history_biography      | 800          | 200          |
| mystery_thriller_crime | 800          | 200          |
| poetry                 | 800          | 200          |
| romance                | 800          | 200          |
| young_adult            | 800          | 200          |
| <b>Total</b>           | <b>6 400</b> | <b>1 600</b> |

Table 3: Hyperparameters

| Parameter                     | Value                       |
|-------------------------------|-----------------------------|
| Base model                    | roberta-base                |
| Epochs                        | 3                           |
| Train batch size (per device) | 10                          |
| Eval batch size (per device)  | 16                          |
| Learning rate                 | $2 \times 10^{-5}$          |
| Warmup steps                  | 100                         |
| Weight decay                  | 0.01                        |
| Max sequence length           | 512 tokens                  |
| Evaluation strategy           | Every 100 steps             |
| Best-model selection          | load_best_model_at_end=True |
| Metric for best model         | accuracy                    |

Training logs (per-step loss and accuracy) are written to `logs/train_steps.log`. A plain-text application log is maintained in `logs/train.log`.

## 5 Evaluation Results

### 5.1 Overall Metrics

Table 4: Evaluation metrics on the held-out test set (1 600 samples)

| Metric      | Local checkpoint | HF Hub model       |
|-------------|------------------|--------------------|
| Accuracy    | 0.6169           | 0.6169 (identical) |
| Eval loss   | 1.1207           | 1.1207             |
| Macro F1    | 0.61             | 0.61               |
| Weighted F1 | 0.61             | 0.61               |

The local and Hub checkpoints produce identical metrics because they are the same saved weights; this confirms correct serialisation and upload.

## 5.2 Per-Class Performance

Table 5: Per-class precision, recall, and F1

| Genre                  | Precision | Recall | F1   |
|------------------------|-----------|--------|------|
| children               | 0.61      | 0.78   | 0.68 |
| comics_graphic         | 0.88      | 0.70   | 0.78 |
| fantasy_paranormal     | 0.58      | 0.34   | 0.43 |
| history_biography      | 0.60      | 0.57   | 0.58 |
| mystery_thriller_crime | 0.52      | 0.60   | 0.56 |
| poetry                 | 0.79      | 0.78   | 0.79 |
| romance                | 0.56      | 0.68   | 0.62 |
| young_adult            | 0.46      | 0.48   | 0.47 |
| Macro avg              | 0.63      | 0.62   | 0.61 |

### Observations:

- *comics\_graphic* and *poetry* are the easiest classes ( $F1 \approx 0.79$ ), likely because their vocabulary is strongly genre-specific.
- *fantasy\_paranormal* and *young\_adult* are the hardest ( $F1 \approx 0.43$ – $0.47$ ), plausibly because both genres share similar tropes and narrative styles.
- Confusion is highest between *romance* and *young\_adult*, and between *fantasy\_paranormal* and *young\_adult*, as shown in the misclassification heatmap.

## 5.3 Confusion Matrix

## 5.4 Per-Class F1 Bar Chart

## 5.5 Misclassification Heatmap

# 6 Docker Deployment

Two Docker images implement the pipeline:

**genre-train** (Dockerfile) Development image for interactive training. The project directory is bind-mounted (`-v $(pwd):/app`) so all outputs (model, logs, results) persist on the host file system.

**genre-eval** (Dockerfile.eval) Minimal production image. The model is *not* baked in; it is fetched from the Hugging Face Hub at runtime via the `HF_REPO` environment variable. Evaluation runs automatically on container start.

Key design decisions:

- `VOLUME` instructions were deliberately omitted — Docker’s anonymous volumes for declared paths override bind-mount subdirectories, making output files invisible on the host. Explicit `-v` flags at `docker run` time are used instead.
- `python:3.11-slim` base image keeps the image lean while supporting all required pip packages.

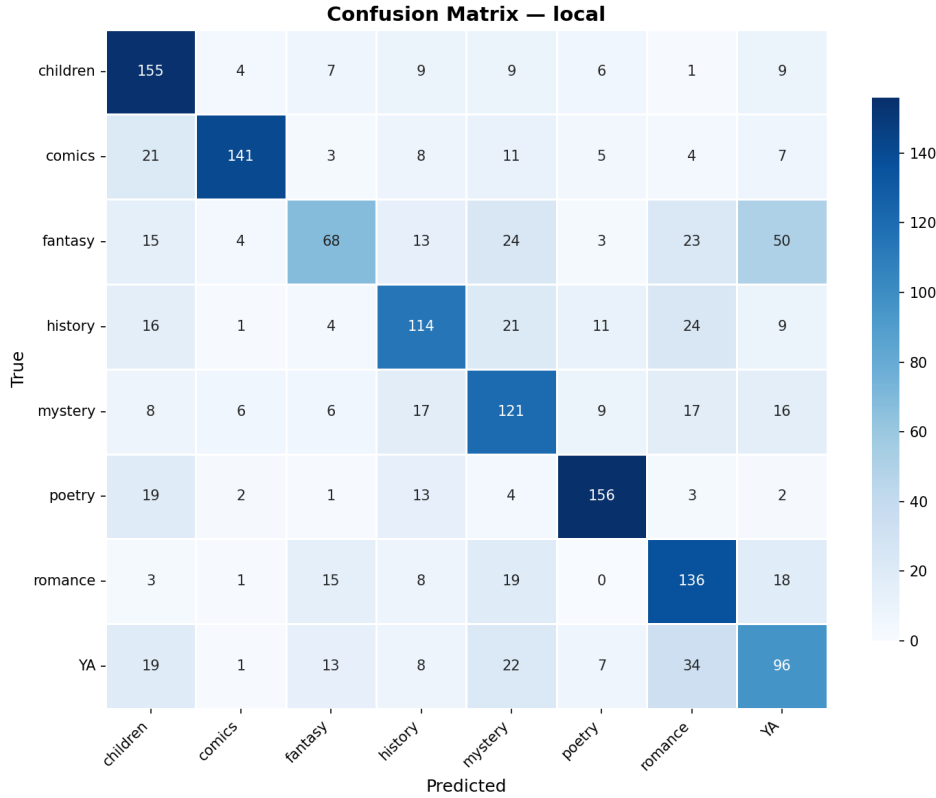


Figure 1: Confusion matrix — local checkpoint (counts)

- `--network=host` enables direct Hugging Face Hub downloads without NAT latency.

## 7 Challenges & Solutions

## 8 Conclusion

A complete end-to-end MLOps pipeline was implemented:

1. The GoodReads dataset was downloaded, cached, and split into 6 400 training and 1 600 test samples across 8 genres.
2. `roberta-base` was fine-tuned for 3 epochs using the Hugging Face `Trainer` API, achieving **61.7% accuracy** (macro F1 = 0.61) on the test set.
3. The model was saved locally and pushed to the Hugging Face Hub.
4. Re-evaluation from the Hub confirmed identical metrics, validating the serialisation pipeline.
5. Two Docker images encapsulate the full workflow: a training image and a production evaluation image.

Accuracy can be improved by: (a) training for more epochs (4–5), (b) switching to `microsoft/deberta-v3-small` once `protobuf` is available in the Docker image, or (c) increasing the per-genre review sample.

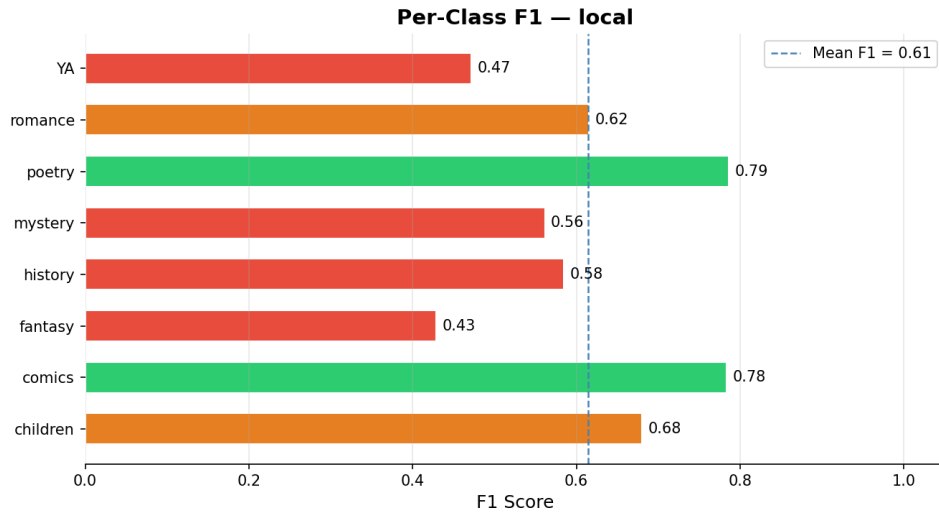


Figure 2: Per-class F1 scores. Colour key: green  $\geq 0.75$ , orange  $\geq 0.60$ , red  $< 0.60$ .

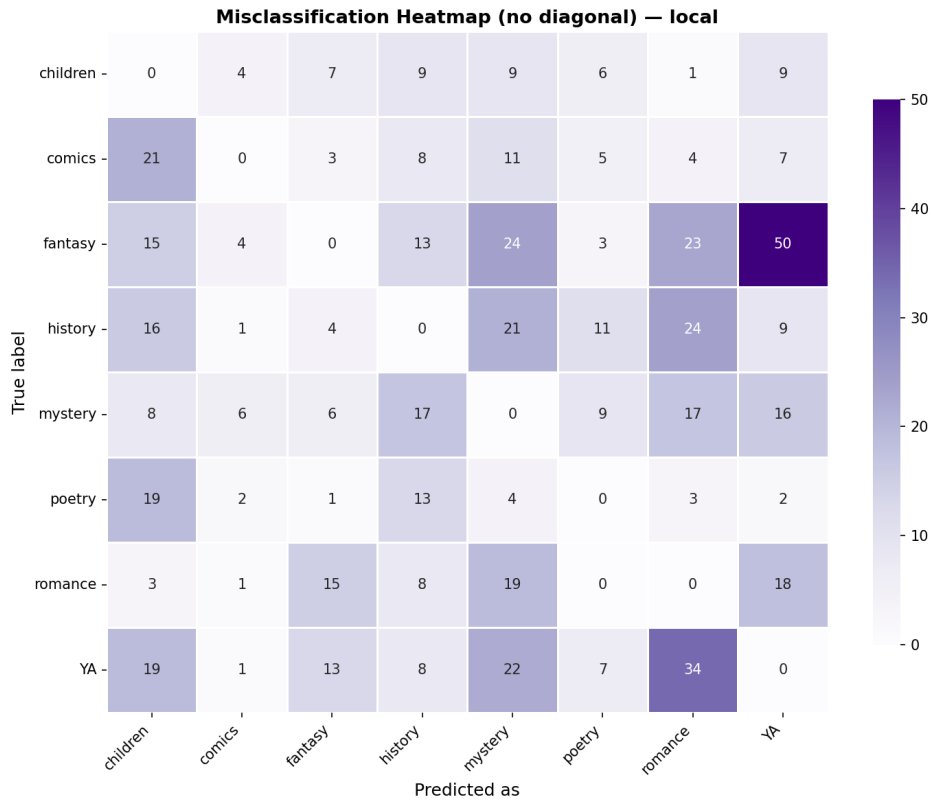


Figure 3: Misclassification heatmap (diagonal zeroed) — highlights confused genre pairs

| Challenge                                                                                                                     | Solution                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>VOLUME</code> in Dockerfile making log/result files invisible on host                                                   | Removed <code>VOLUME</code> declarations; use bind-mount ( <code>-v \$(pwd) : /app</code> ) instead.                                                                |
| <code>trainer.push_to_hub(repo_id)</code> silently treating the argument as a commit message $\rightarrow$ model not uploaded | Changed to <code>model.push_to_hub(repo_id)</code> and <code>tokenizer.push_to_hub(repo_id)</code> which correctly accept the repo id.                              |
| <code>PlainTextLogCallback.on_log()</code> raising <code>ValueError: I/O operation on closed file</code>                      | <code>f.flush()</code> was outside the <code>with open()</code> block. Moved inside.                                                                                |
| <code>microsoft/deberta-v3-small</code> failing with <code>ImportError: protobuf not found</code>                             | Added <code>protobuf&gt;=4.25.0</code> to <code>requirements.txt</code> ; switched default to <code>roberta-base</code> (no extra deps) until the image is rebuilt. |
| Large dataset download ( $8 \times$ gzip HTTP streams, $\sim 500$ MB)                                                         | Added <code>pickle</code> cache ( <code>genre_reviews_dict.pickle</code> ) — data downloaded only once; subsequent runs restore from disk in $<1$ s.                |
| Jupyter magic ( <code>%matplotlib inline</code> ) present in notebook cells                                                   | Removed in <code>.py</code> scripts; used <code>matplotlib.use('Agg')</code> for headless (Docker-compatible) plot rendering.                                       |

Table 6: Challenges encountered and their resolutions